

MACHINE LEARNING MODEL DEVELOPMENT AND EVALUATION GROUP 5 ASSIGNMENT

Group Members

Khaemba Imelda Nasimiyu - ST62/55266/2025

Grace Mwai - ST62/56407/2025

Murangiri Julius Elijah - ST62/56956/2025

Frieze Wandabwa - ST62/55175/2025

Mbarak Abdallah Kumbo - ST62/55566/2025

Seth Makori - ST62/55318/2025

Victor Chiuri Gateru - ST62/80979/2024

Paul Maweu Manthi - ST62/56199/2025

Erick Ngari - ST62/56218/2025

Framul Muriithi - ST62/55824/2025

Kariuki James Kariuki - ST62/55039/2025

Dataset Used

This study uses the **Crop Recommendation Dataset** obtained from Kaggle. The dataset contains soil nutrients and environmental conditions used to determine suitable crops for farming.

Dataset Features

Feature	Description
N	Nitrogen content in soil
P	Phosphorus content
K	Potassium content
temperature	Temperature (°C)
humidity	Humidity (%)
ph	Soil pH level
rainfall	Rainfall (mm)
label	Crop type

Preprocessing

The dataset was converted into a binary classification problem by selecting **rice as the target class**, where:

- 1 = Suitable for rice
- 0 = Not suitable for rice

The dataset provides a realistic representation of agricultural conditions relevant to regions such as Kenya.

QUESTION 1: Logistic Regression – Crop Suitability (Using Real Dataset)

1.1 Python Implementation

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


# Load dataset

data = pd.read_csv("Crop_recommendation.csv")


# Convert to binary classification (rice vs others)

data['target'] = data['label'].apply(lambda x: 1 if x == 'rice' else 0)


# Features and target

X = data[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]

y = data['target']


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Normalize features

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
```

```
# Train model

model = LogisticRegression()

model.fit(X_train, y_train)


# Predictions

y_pred = model.predict(X_test)


# Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Precision:", precision_score(y_test, y_pred))

print("Recall:", recall_score(y_test, y_pred))

print("F1-score:", f1_score(y_test, y_pred))
```

1.2 Methodology

- The dataset was loaded from a CSV file using pandas
- The target variable was converted into binary (rice = 1, others = 0)
- Features were selected from soil nutrients and environmental conditions
- Data was split into training and testing sets (80/20)
- Features were normalized using StandardScaler
- Logistic Regression model was trained and evaluated

1.3 Metrics Explanation

- **Accuracy:** Overall correctness of predictions
- **Precision:** Correctness of predicted rice cases
- **Recall:** Ability to detect actual rice conditions
- **F1-score:** Balance between precision and recall

1.4 Interpretation

The model performs well in predicting whether environmental conditions are suitable for rice. A high recall indicates that most suitable conditions are correctly identified, which is important in agricultural decision-making.

1.5 Conclusion

Logistic Regression is effective for crop suitability prediction due to its simplicity and good performance in binary classification tasks.

QUESTION 2: Decision Tree – Crop Suitability

2.1 Python Implementation

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


# Load dataset

data = pd.read_csv("Crop_recommendation.csv")


# Convert to binary classification (rice vs others)

data['target'] = data['label'].apply(lambda x: 1 if x == 'rice' else 0)


# Features and target

X = data[['N','P','K','temperature','humidity','ph','rainfall']]

y = data['target']


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Train Decision Tree model

model = DecisionTreeClassifier(max_depth=4)

model.fit(X_train, y_train)


# Predictions

y_pred = model.predict(X_test)


# Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Precision:", precision_score(y_test, y_pred))

print("Recall:", recall_score(y_test, y_pred))

print("F1-score:", f1_score(y_test, y_pred))
```

2.2 Methodology

- The dataset was loaded and converted into binary classification (rice vs others)
- Relevant features (soil nutrients and environmental factors) were selected
- Data was split into training and testing sets
- A Decision Tree model was trained with a limited depth to reduce overfitting

2.3 Metrics Explanation

- **Accuracy:** Overall correctness of predictions
- **Precision:** Correct prediction of rice suitability
- **Recall:** Ability to detect actual suitable conditions
- **F1-score:** Balance between precision and recall

2.4 Interpretation

The model performs well in predicting crop suitability. Features such as soil nutrients and environmental conditions strongly influence the outcome. Limiting the tree depth helps prevent overfitting.

2.5 Conclusion

Decision Trees are effective for agricultural classification problems due to their interpretability, but require tuning to improve generalization.

QUESTION 3: KNN – Crop Suitability Prediction

3.1 Python Implementation

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score


# Load dataset
data = pd.read_csv("Crop_recommendation.csv")


# Convert to binary classification (rice vs others)
data['target'] = data['label'].apply(lambda x: 1 if x == 'rice' else 0)


# Select features (as required: age/usage equivalent → environmental features)
X = data[['temperature', 'humidity']]
y = data['target']


# Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)


# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# Apply KNN (k = 5, Euclidean distance)
model = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
```

3.2 Methodology

- The dataset was loaded and converted into binary classification (rice vs others)
- Selected features: **temperature and humidity**
- Data was normalized using StandardScaler to ensure equal scaling
- Dataset was split into training and testing sets
- KNN was applied with **k = 5 using Euclidean distance**

3.3 Metrics Explanation

- **Accuracy:** Overall correctness of predictions
- **Precision:** Correctness of predicted suitable conditions
- **Recall:** Ability to detect actual suitable conditions

3.4 Interpretation

The model performs well after normalization since KNN relies on distance calculations. Using $k = 5$ provides a balance between overfitting and underfitting, leading to stable predictions.

3.5 Conclusion

KNN is effective for crop suitability prediction when data is properly scaled, but its performance depends on feature selection and the choice of k.

QUESTION 4: SVM + Kernel Comparison + Hyperparameter Tuning

4.1 Python Implementation

```
import pandas as pd

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


# Load dataset

data = pd.read_csv("Crop_recommendation.csv")


# Convert to binary classification (rice vs others)

data['target'] = data['label'].apply(lambda x: 1 if x == 'rice' else 0)


# Features and target

X = data[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]

y = data['target']


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Normalize data (important for SVM)

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Grid Search for best parameters
param_grid = {
    'C': [0.1, 1, 10],
    'gamma': [1, 0.1, 0.01],
    'kernel': ['linear', 'poly', 'rbf']
}

grid = GridSearchCV(SVC(), param_grid, cv=3)
grid.fit(X_train, y_train)

# Best model
model = grid.best_estimator_

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Best Parameters:", grid.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-score:", f1_score(y_test, y_pred))
```

4.2 Methodology

- The dataset was loaded and converted into binary classification (rice vs others)
- Features were selected and normalized using StandardScaler
- Data was split into training and testing sets
- SVM models with **linear, polynomial, and RBF kernels** were trained
- GridSearchCV was used to find optimal values of **C and gamma**
- Model performance was evaluated using standard classification metrics

4.3 Metrics Explanation

- **Accuracy:** Overall correctness
- **Precision:** Correct positive predictions
- **Recall:** Ability to detect actual positives
- **F1-score:** Balance between precision and recall

4.4 Interpretation

The **RBF kernel performs best** because it can model non-linear relationships between environmental conditions and crop suitability. Linear kernel performs well only when data is linearly separable, while polynomial captures some complexity but may overfit.

4.5 Conclusion

SVM with RBF kernel provides the best performance for this dataset. Hyperparameter tuning (C and gamma) significantly improves model accuracy and generalization.

QUESTION 5: Decision Tree vs Random Forest

5.1 Python Implementation

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, f1_score
```

```
# Load dataset

data = pd.read_csv("Crop_recommendation.csv")


# Convert to binary classification (rice vs others)

data['target'] = data['label'].apply(lambda x: 1 if x == 'rice' else 0)


# Features and target

X = data[['N','P','K','temperature','humidity','ph','rainfall']]
y = data['target']


# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Decision Tree

tree = DecisionTreeClassifier(max_depth=5)

tree.fit(X_train, y_train)

y_pred_tree = tree.predict(X_test)


# Random Forest

rf = RandomForestClassifier(n_estimators=100, max_depth=5)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)


# Evaluation

print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_tree))

print("Decision Tree F1:", f1_score(y_test, y_pred_tree))
```

```
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))  
print("Random Forest F1:", f1_score(y_test, y_pred_rf))
```

5.2 Methodology

- The same dataset was used for both models to ensure consistency
- Data was split into training and testing sets
- A Decision Tree and Random Forest model were trained
- Performance was compared using accuracy and F1-score

5.3 Metrics Explanation

- **Accuracy:** Overall correctness
- **F1-score:** Balance between precision and recall

5.4 Interpretation

Random Forest performs better than Decision Tree because it combines multiple trees, reducing overfitting and improving generalization. Decision Trees tend to overfit when used alone.

5.5 Hyperparameters Impact

- **max_depth:** Controls tree complexity and prevents overfitting
- **n_estimators:** Number of trees in Random Forest (more trees improve stability and accuracy)

5.6 Conclusion

Random Forest outperforms Decision Tree due to ensemble learning, making it more accurate and reliable for crop suitability prediction.

